

Linear Classifiers and Logistic Regression

Gautam Kamath

A (very) simple linear classifier

- (Draw pass class example)
- Plausible grading scheme: if average of hw and exams > 0.5 , pass.
- Equivalently: if $0.5 \cdot \text{homework} + 0.5 \cdot \text{exams} > 0.5$, pass.
 - Or: if $0.5 \cdot \text{homework} + 0.5 \cdot \text{exams} - 0.5 > 0$, pass.
- Rewrite as: $\text{sign}(\langle (0.5, 0.5), (\text{homework}, \text{exams}) \rangle - 0.5)$
 - Dot product notation: $\langle u, v \rangle = \sum_i u_i v_i$
 - $\text{sign}(a) = 1$ if $a > 0$, $= -1$ otherwise
- Let $x = (\text{homework}, \text{exams})$: $y = \text{sign}(\langle (0.5, 0.5), x \rangle - 0.5)$

Linear classifiers

- A very general concept
- Score $s(x) = \langle w, x \rangle + b$, weight w and bias b
- Predict $h(x) = \text{sign}(s(x))$
- Geometric intuition:
 - $\langle w, x \rangle + b$ is the decision boundary
 - Vector w is the normal to the hyperplane
- Correct classification when $y = h(x)$

A few notational tricks

- Absorb the bias
 - Define $\tilde{w} = [w, b]$ and $\tilde{x} = [x, 1]$
 - Then $\langle w, x \rangle + b = \langle \tilde{w}, \tilde{x} \rangle$
 - “Padding trick”
 - We’ll drop the tildes to simplify notation
- Multiply to check if sign of prediction matches label
 - $y = \text{sign}(\langle w, x \rangle) \iff y \cdot \text{sign}(\langle w, x \rangle) \geq 0$
 - If correct classification, $y = \text{sign}(\langle w, x \rangle) = 1$ or -1 , so $y \cdot \text{sign}(\langle w, x \rangle) = 1$
 - If incorrect classification (signs differ), $y \cdot \text{sign}(\langle w, x \rangle) = -1$

Perceptron

- Assume training data in which the positive and negative examples are “linearly separable”
 - There exists w such that $y_i \langle w, x_i \rangle > 0$
 - (Draw picture)
- If data is linearly separable, then the perceptron algorithm finds a separating classifier
 - Finds $y_i = \text{sign}(\langle w, x_i \rangle)$ for all $i \in [n]$
 - Algorithm: while $\exists i$ s.t. $y_i \langle w, x_i \rangle \leq 0$
 - “While there exists a point which is incorrectly classified”
 - then $w \leftarrow w + y_i x_i$
 - “Update the weights”

Limitations of Perceptron (draw)

- May return any hyperplane, doesn't prefer a "large margin" solution
 - Inductive bias of the learning algorithm would help...
- No guarantees for datasets which are not linearly separable
 - The algorithm will not converge
 - Severely limiting, data is rarely linearly separable

Transitioning to logistic regression

- Keep:
 - Linear decision boundary
 - Classifier will still be $h(x) = \text{sign}(\langle w, x \rangle)$
- Fix:
 - Instead of “hard” 0/1 predictions, model probabilities
 - Handle data that is not linearly separable
- Note: changing from $y \in \{\pm 1\}$ to $y \in \{0,1\}$
 - Will make some algebra easier

Intuition

- Model the conditional probability $\eta(x) = \Pr[Y = 1 \mid X = x]$
 - The “regression function”
 - Perceptron had no model for $\eta(x)$, made “hard” decisions instead
 - Can still give binary prediction by thresholding at 0.5
- Deriving a reasonable model of data generation
- (Draw hours studied vs passed class example)
- (Draw perceptron example, sharp threshold, boundary examples)
- Bernoulli model: parameterize probability of label y by feature vector x , parameter vector w
 - $\Pr[y = 1 \mid x, w] = p(x, w) \in [0, 1]$
 - $\Pr[y = 0 \mid x, w] = 1 - p(x, w)$

How to parameterize?

- $\Pr[y = 1 \mid x, w] = p(x, w) \in [0,1]$. How do we define $p(x, w)$?
- Take 1: $p(x, w) = \langle x, w \rangle$
 - Being far on the positive side of the hyperplane makes it large, vice versa
 - Why doesn't it work? LHS is in $[0,1]$, while RHS is over $\mathbf{R}$
- Take 2: $\log \left(\frac{p(x, w)}{1 - p(x, w)} \right) = \langle x, w \rangle$ (“logit transform”)
 - Assume the log-odds are linear in the features
 - Sanity check: $p(x, w)$ ranging over $[0,1]$ causes LHS to range over $\mathbf{R}$ (like RHS)

Rearranging the parameterization

$$\log \left(\frac{p(x, w)}{1 - p(x, w)} \right) = \langle x, w \rangle$$

$$\frac{p(x, w)}{1 - p(x, w)} = \exp(\langle x, w \rangle) \text{ (LHS: "odds" – mention odds ratio interp.)}$$

$$p(x, w) = \exp(\langle x, w \rangle) (1 - p(x, w))$$

$$p(x, w) = \exp(\langle x, w \rangle) - \exp(\langle x, w \rangle) p(x, w)$$

$$p(x, w)(1 + \exp(\langle x, w \rangle)) = \exp(\langle x, w \rangle)$$

$$p(x, w) = \frac{\exp(\langle x, w \rangle)}{1 + \exp(\langle x, w \rangle)}$$

$$p(x, w) = \frac{1}{1 + \exp(-\langle x, w \rangle)} \triangleq \text{sigmoid}(\langle x, w \rangle)$$

(draw sigmoid)

Visualizing $p(x, w)$

- $p(x, w) = \frac{1}{1 + \exp(-\langle x, w \rangle)} \triangleq \text{sigmoid}(\langle x, w \rangle)$
- (Draw linear separator, point on line $\langle x, w \rangle = 0$, point on either side)
- (Draw picture from the side of sigmoid)
- Why sigmoid? Admittedly a bit arbitrary
- Any monotone function from $\mathbf{R} \rightarrow [0,1]$ works
- E.g., take $\text{sign}(\langle x, w \rangle)$ and you recover perceptron

Predicting using $p(x, w)$

- $p(x, w) = \frac{1}{1 + \exp(-\langle x, w \rangle)}$
- If $p(x, w) > \frac{1}{2}$, predict $\hat{y} = 1$. Otherwise, predict $\hat{y} = 0$.
- What happens if w is replaced by $10w$?
- Prediction rule corresponds to $\hat{y} = \text{sign}(\langle x, w \rangle)$, same as perceptron!
 - Difference 1: While we use same predictions, we optimize different functions
 - E.g., this can handle non linearly separable, couldn't with perceptron
 - Difference 2: Magnitude of $p(x, w)$ indicates model's conditional probability (a type of “confidence”) in prediction
 - Hence why we call it *regression* – we're learning conditional probabilities, which imply predictions

Summary thus far

- We have come up with a reasonable way to *model* a linear classification problem using logistic regression
 - Given feature vector x , the model uses a parameter vector w to approximate the conditional class probability $\eta(x)$, used to derive a prediction y
- Remaining question: how do we *learn* the parameter vector w ?
 1. Framing this as an optimization problem
 2. Solving the optimization problem

Modeling to Learning

- Setup: Given $(x_1, y_1), \dots, (x_n, y_n) \sim_{i.i.d.} P$
- Problem defined by a *loss function* $\ell_w(x, y)$
 - Sometimes written as $\ell(w, x, y)$. w is the *parameter vector*.
- Goal: output $\arg \min_w \mathbf{E}_{(x,y) \sim P} [\ell_w(x, y)]$
 - Parameter vector w which minimizes loss given new point from distribution
- E.g., for perceptron:
 - $\ell_w(x, y) = 0$ if $\text{sign}(\langle w, x \rangle) = y$, $\ell_w(x, y) = 1$ if $\text{sign}(\langle w, x \rangle) \neq y$
 - Goal: output $\arg \min_w \mathbf{E}_{(x,y) \sim P} [\ell_w(x, y)] = \arg \min_w \Pr_{(x,y) \sim P} [\text{sign}(\langle w, x \rangle) \neq y]$

Empirical Risk Minimization (ERM)

- Population risk: $R(w) = \mathbf{E}_{(x,y) \sim P} [\ell_w(x, y)]$
 - Goal: output $\arg \min_w R(w)$
 - But we don't know the distribution P – we only have (x_i, y_i) 's from P
 - What do we do?
- Minimize the expected loss over the *training dataset*
 - i.e., the *empirical* distribution
- Empirical risk: $\hat{R}_n(w) = \frac{1}{n} \sum_{i=1}^n \ell_w(x_i, y_i)$
- Empirical risk minimization (ERM): $\hat{w} = \arg \min_w \hat{R}_n(w)$
 - Goal is to find w which minimizes average loss function
- Which loss function do we choose?

Maximum Likelihood Estimation (MLE)

- Maximum Likelihood principle: find model parameters which maximize the probability of the observed data
- $\arg \max_{\text{model parameters}} \Pr[\text{observed data} \mid \text{model parameters}]$
- Needs some probabilistic model on observed data wrt model parameters
 - i.e., $(x, y) \sim P_w$, where w are the model parameters
- $\hat{w} = \arg \max_w \prod_i P_w(y_i \mid x_i)$
- What is our modeling assumption?
 - $\Pr[Y = 1 \mid X = x] = p(x, w)$

Deriving the MLE parameter vector

$$\begin{aligned}\hat{w} &= \arg \max_w \prod_{i=1}^n \Pr[y_i | w, x_i] \\&= \arg \max_w \prod_{i=1}^n p(x_i, w)^{y_i} (1 - p(x_i, w))^{1-y_i} \\&\quad (\text{Let } p_i = p(x_i, w)) \\&= \arg \max_w \log \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i} \\&= \arg \max_w \sum_{i=1}^n \log(p_i^{y_i} (1 - p_i)^{1-y_i}) \\&= \arg \max_w \sum_{i=1}^n y_i \log p_i + (1 - y_i) \log(1 - p_i) \\&= \arg \min_w -\frac{1}{n} \sum_{i=1}^n y_i \log p_i + (1 - y_i) \log(1 - p_i)\end{aligned}$$

Interlude: Cross-Entropy Loss

- Cross-entropy loss: $\ell(p, y) = -y \log p - (1 - y) \log(1 - p)$
- If $y = 1$, $\ell(p, 1) = -\log p$. If $y = 0$, $\ell(p, 0) = -\log(1 - p)$.
- Say $y = 1$ and:
 - $p = 1$? $p = 0.5$? $p = 0$?
- $\mathbf{E}[\ell(p, y) | X = x] = -\eta(x) \log p - (1 - \eta(x)) \log(1 - p)$
 - What value of p minimizes this?
 - Setting derivative equal to 0:
 - $-\frac{\eta(x)}{p} + \frac{1-\eta(x)}{1-p} = 0$
 - $p = \eta(x)$
- Expected loss is minimized when probability is regression function $\eta(x)$

Deriving the MLE parameter vector (cont)

Recall: $p(x, w) = \frac{1}{1 + \exp(-\langle x, w \rangle)}$.

Suppose some $y_i = 1$. Then

$$\begin{aligned} y_i \log p_i + (1 - y_i) \log(1 - p_i) &= \log p_i \\ &= \log(1 + \exp(-\langle x_i, w \rangle))^{-1} \\ &= -\log(1 + \exp(-\langle x_i, w \rangle)) \end{aligned}$$

Similarly, if $y_i = 0$, then

$$\begin{aligned} y_i \log p_i + (1 - y_i) \log(1 - p_i) &= \log(1 - p_i) \\ &= \log\left(\frac{\exp(-\langle x_i, w \rangle)}{1 + \exp(-\langle x_i, w \rangle)}\right) \\ &= -\log(1 + \exp(\langle x_i, w \rangle)) \end{aligned}$$

Deriving the MLE parameter vector

$$\hat{w} = \arg \min_w -\frac{1}{n} \sum_{i=1}^n y_i \log p_i + (1 - y_i) \log(1 - p_i)$$

If $y_i = 1$, argument is $-\log(1 + \exp(-\langle x_i, w \rangle))$

If $y_i = 0$, argument is $-\log(1 + \exp(\langle x_i, w \rangle))$

$$\hat{w} = \arg \min_w \frac{1}{n} \sum_{i=1}^n \log(\exp(-y_i \langle x_i, w \rangle) + \exp((1 - y_i) \langle x_i, w \rangle))$$

An equivalent formulation

$$\hat{w} = \arg \min_w \frac{1}{n} \sum_{i=1}^n \log(\exp(-y_i \langle x_i, w \rangle) + \exp((1 - y_i) \langle x_i, w \rangle))$$

Let $\tilde{y}_i = +1$ if $y_i = 1$, and $\tilde{y}_i = -1$ if $y_i = 0$.

$$\hat{w} = \arg \min_w \frac{1}{n} \sum_{i=1}^n \log(1 + \exp(-\tilde{y}_i \langle x_i, w \rangle))$$

Calculus Reminder

- Derivative

- Let $f(x) : \mathbf{R} \rightarrow \mathbf{R}$ be a scalar-valued function of one variable
- Derivative $f'(x) = \frac{df}{dx} : \mathbf{R} \rightarrow \mathbf{R}$

- Gradient

- Let $f(v) : \mathbf{R}^d \rightarrow \mathbf{R}$ be a scalar-valued function of d variables
- Gradient $\nabla f(v) = \left(\frac{\partial f}{\partial v_1}, \dots, \frac{\partial f}{\partial v_d} \right) : \mathbf{R}^d \rightarrow \mathbf{R}^d$

Calculus Reminder

- Hessian
 - Let $f(v) : \mathbf{R}^d \rightarrow \mathbf{R}$ be a scalar-valued function of d variables
 - Hessian $\nabla^2 f(v) : \mathbf{R}^d \rightarrow \mathbf{R}^{d \times d}$

$$\begin{bmatrix} \frac{\partial^2 f}{\partial v_1^2} & \cdots & \frac{\partial^2 f}{\partial v_1 \partial v_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial v_1 \partial v_d} & \cdots & \frac{\partial^2 f}{\partial v_d^2} \end{bmatrix}$$

Convexity and Optimization

- How do we pick a good loss function?
 - Depends on structure we assume in data, consider e.g., perceptron
 - Also may depend on convenience, especially for optimization
- (Draw picture of convex function)
- Function f is convex iff for all $\lambda \in [0,1], x_1, x_2$,
$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$
- Alternatively: $f''(x) \geq 0$ (1D functions) or $\nabla^2 f(x) \succcurlyeq 0$
 - Matrix $M \in \mathbf{R}^{d \times d}$ is *positive semidefinite* (PSD) iff $v^T M v \geq 0$ for all vectors $v \in \mathbf{R}^d$
 - Also written $M \succcurlyeq 0$
- (Draw non-convex function, local, global min, saddle point)

Convexity

- Convexity is nice because it makes optimization easier
- Given a convex function f , if $\nabla f(w) = \vec{0}$, then w is a *global minimizer*
- Tying back to ERM: goal is to find $\arg \min_w \frac{1}{n} \sum_{i=1}^n \ell_w(x_i, y_i)$
- If ℓ_w is convex (in w), then ERM is equivalent to finding w^* such that

$$\nabla_w \frac{1}{n} \sum_{i=1}^n \ell_{w^*}(x_i, y_i) = \frac{1}{n} \sum_{i=1}^n \nabla_w \ell_{w^*}(x_i, y_i) = \vec{0}$$

Logistic regression gradient

- $\ell_w(x_i, y_i) = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$
 - $p_i = p(x_i, w) = \frac{1}{1 + \exp(-\langle x_i, w \rangle)}$
- $\nabla_w p_i = \nabla_w (1 + \exp(-\langle x_i, w \rangle))^{-1}$
 - $= -(1 + \exp(-\langle x_i, w \rangle))^{-2} \nabla_w (1 + \exp(-\langle x_i, w \rangle))$
- $\nabla_w (1 + \exp(-\langle x_i, w \rangle)) = -x_i \exp(-\langle x_i, w \rangle)$
- $\nabla_w p_i = \frac{x_i \exp(-\langle x_i, w \rangle)}{(1 + \exp(-\langle x_i, w \rangle))^2}$, $p_i = \frac{1}{1 + \exp(-\langle x_i, w \rangle)}$ and $1 - p_i = \frac{\exp(-\langle x_i, w \rangle)}{1 + \exp(-\langle x_i, w \rangle)}$
- Thus $\nabla_w p_i = p_i(1 - p_i)x_i$

Logistic regression gradient

- $\ell_w(x_i, y_i) = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$
- $\nabla_w p_i = p_i(1 - p_i)x_i$
- $\nabla_w \ell_w(x_i, y_i) = -y_i \cdot \frac{1}{p_i} \cdot \nabla_w p_i - (1 - y_i) \cdot -\frac{1}{1-p_i} \cdot \nabla_w p_i$
- $= \left(-\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i} \right) p_i(1 - p_i)x_i$
- $= (p_i(1 - y_i) - y_i(1 - p_i))x_i = (p_i - y_i)x_i$
- $\nabla_w \ell_w(x_i, y_i) = (p_i - y_i)x_i$
 - Gradient is a scaling of the feature vector x_i
 - If $y_i = 1$ and $p_i \approx 1$, then a small gradient
 - If $y_i = 1$ and $p_i \approx 0$, then a large gradient

Logistic regression's loss is convex

- $\nabla_w \ell_w(x_i, y_i) = (p_i - y_i)x_i$
- $\nabla_w^2 \ell_w(x_i, y_i) = \nabla_w(p_i - y_i)x_i = x_i(\nabla_w p_i)^T = p_i(1 - p_i)x_i x_i^T$
- Have to show that $p_i(1 - p_i)x_i x_i^T$ is PSD
 - $v^T(p_i(1 - p_i)x_i x_i^T)v \geq 0$ for all vectors v
- $p_i(1 - p_i)(v^T x_i)(x_i^T v) = p_i(1 - p_i)\langle x_i, v \rangle^2 \geq 0$
- Since the Hessian of the loss function is PSD, logistic regression is convex

Summary of logistic regression loss

- $\ell_w(x_i, y_i) = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$
- Loss function ℓ_w is convex
- Gradient $\nabla_w \ell_w(x_i, y_i) = (p_i - y_i)x_i$
- Goal is to find a vector w such that $\sum_i \nabla_w \ell_w(x_i, y_i) = 0$
- No closed form solution (i.e., a w that you can just solve for)
- Have to turn to iterative methods for optimization